CkaDinTi
Arron's Project Tasks

Owner: teacher desktop, lesson ingestion, AI generation, quiz review, and teacher reports

Build the trusted teacher-side workflow from lesson import through approved quiz and game setup.

Project foundation

The product is a desktop-first, local-first AI classroom quiz platform. The supplied deep-research PDF is the product brief; this document is the actionable task assignment for this team member.

Shared technology: Electron + React + TypeScript for the teacher app; React + Vite for students; SQLite + Drizzle; Fastify + Socket.IO; Zod contracts; OpenAI structured outputs with mandatory teacher review.

Milestone A - teacher foundation

- Scaffold the pnpm workspace and Electron Forge teacher app.
- Create secure Electron main/preload boundaries; keep secrets and filesystem access out of the renderer.
- Implement SQLite/Drizzle repositories for lessons, lesson blocks, quizzes, questions, options, evidence, and AI jobs.
- Define Zod contracts for lesson import, quiz generation, review state, and teacher IPC.

Milestone B - content pipeline

- Support TXT, Markdown, text-based PDF, and DOCX ingestion with normalized text and source evidence ranges.
- Add content hashing, deduplication, validation errors, and parser fixtures.
- Implement OpenAI Responses API generation with strict structured output, prompt versioning, evidence, explanation, difficulty, topic, and confidence label.
- Keep teacher approval mandatory; generated output is never publishable by itself.

Milestone C - teacher UX

- Build Dashboard, Lessons, Quizzes, Review, Reports, and Settings screens.
- Implement edit, delete, reorder, duplicate, add manually, regenerate, approve/reject, and approve-all actions.
- Add source-evidence view-in-lesson behavior.

- Build teacher host/lobby screens against Edin's shared contracts without implementing the game server.

Acceptance criteria

- A lesson can be imported, normalized, generated into schema-valid questions, reviewed, edited, and approved locally.
- Approved quiz data can be exported through shared contracts for a host session.
- No OpenAI key appears in renderer bundles or student payloads.
- Tests cover parsers, schema validation, review transitions, and persistence.

Non-overlap rules

- Do not implement Fastify/Socket.IO server logic, authoritative scoring, or student gameplay; Edin owns those areas.
- Own lesson/quiz database tables. Coordinate any migration that touches Edin's session/result tables.
- Publish approved quiz snapshots through packages/contracts; do not duplicate schemas.
- If using Graphify, keep it behind a teacher-report adapter and never use it for scoring or persistence.

Workflow

- Work only on the matching branch: Arron.
- Use RTK wrappers for repository inspection and test output: rtk git, rtk rg, rtk read, and rtk test.
- Keep commits focused and prefix feature commits with the owner name.
- Update docs/IMPLEMENTATION_STATUS.md when a milestone is completed.
- Never commit API keys, student personal data, or generated SQLite files.

References: Graphify (optional visualization adapter) - https://github.com/Graphify-Labs/graphify
RTK (token-efficient CLI workflow) - https://github.com/rtk-ai/rtk
Repository - https://github.com/CkaDinTi/CkaDinTi